VELOOP REWARDS - Earning & Reward Analytics Specification

Data Analyst Internship, Task 01 | Work Package C

Authenticity notice: Every event, field and category below is **Proposed** or **To Be Verified**. No internal failure codes are invented, and no simulated or real VELOOP statistics appear here.

1. Purpose and key principles

This specification defines how VELOOP should measure earning activity from discovery to reward. Four principles drive it:

- **Activity is not reward.** A task being completed and a reward being credited are separate facts, recorded by separate events.
- **A claim is not success.** A user or provider saying a task is done must pass validation before it counts as completed.
- **Failure has a reason.** One task_failed event carries a failure_category property, so reasons can be counted without many event names.
- **External systems are not controlled by VELOOP.** Ads, surveys, offers, games and tracking tools can be late, wrong or silent, and the data model must allow for that.

2. Earning lifecycle measurement (C0)

age	What Happens	Event(s)	Key Fields	Metric (formula only, no targets)	Caution	Status
Discovery	User sees or opens an earning opportunity	task_viewed, task_opened	user_id, task_id, task_type, feature, timestamp	Open rate = users who opened / users who viewed	Define 'viewed' as visible on screen, not merely loaded; views can be high volume	Proposed
Start	User begins the activity	task_started	task_activity_id, task_id, user_id, started_at, source	Start rate = task_started / task_opened	Starting an external task may take the user out of the app; record the source	Proposed
Completion	User claims or finishes the activity	(stored as completion_claimed_at on the task activity record; no separate event)	task_activity_id, completion_claimed_at	Claim rate = completion claims / task_started	A claim is not success. Do not count it as a completed task	Proposed
Validation	System or provider checks the claim	task_completed (passed) or task_failed with failure_category (not passed)	validation_status, validation_completed_at, validator (internal / provider)	Validation pass rate = validated completions / completion claims	Providers may validate late; keep 'pending' distinct from 'failed'	Proposed
Reward	Reward is held or credited to the wallet	reward_pending, reward_credited	reward_id, reward_amount, reward_unit, wallet_transaction_id	Credit rate = validated tasks with a credited reward / validated tasks	Credit can lag and can be reversed later; completion does not equal credit	Proposed

3. Task-level analytics fields (C1)

task_id identifies the task in the catalogue; task_activity_id identifies one attempt by one user. Keeping them apart is what allows repeat attempts, retries and per-attempt failure reasons.

Field	Type	Required?	Definition	Validation	Status
task_activity_id	string	Yes	Unique key for one attempt by one user at one task (a task can be attempted many times)	Unique; immutable	Proposed
task_id	string	Yes	Key of the task in the task catalogue (the definition of the task, not the attempt)	Must exist in task catalogue	To Be Verified
task_type	string (enum)	Yes	Category of task (values to be taken from real VELOOP task types)	Approved list only	To Be Verified
feature	string (enum)	Yes	Product feature that offered the task (e.g. watch ads, missions)	Approved list only	To Be Verified
user_id	string	Yes	Stable user key	Must exist in Users	To Be Verified
started_at	timestamp (UTC)	Yes	When the user started	ISO 8601 UTC; not in the future	Proposed
completion_claimed_at	timestamp (UTC)	Conditional	When the user or provider reported completion	Not before started_at	Proposed
completed_at	timestamp (UTC)	Conditional	When completion passed validation	Required when status = validated; not before completion_claimed_at	Proposed
status	string (enum)	Yes	Current state of the attempt	started, completion_claimed, validated, failed, abandoned (final list to be agreed)	Proposed
validation_status	string (enum)	Yes	Result of validation	not_required, pending, passed, failed	Proposed
validation_completed_at	timestamp (UTC)	Conditional	When validation finished	Not before completion_claimed_at	Proposed
reward_amount	decimal	Conditional	Reward amount tied to this attempt	Not negative; present if a reward exists	Proposed
reward_unit	string (enum)	Conditional	Unit of the reward	Required when reward_amount present; must be a real VELOOP unit	To Be Verified
reward_id	string	Conditional	Link to the reward record	Must exist in Rewards	Proposed
failure_category	string (enum)	Conditional	Why the attempt failed (see C2)	Required when status = failed or abandoned	Proposed
failure_detail_code	string	Optional	VELOOP internal code, only if one exists; not invented here	Free of personal data	To Be Verified
source	string (enum)	Yes	Where the record came from: frontend, backend, provider	Approved list	Proposed
provider_name	string	Optional	External provider involved, if any	Approved provider list	To Be Verified
provider_reference_id	string	Optional	Provider's own id for this activity	Unique per provider	To Be Verified
attempt_number	integer	Optional	Nth attempt by this user at this task	Integer >= 1	Proposed

4. Failed activity classification (C2)

This is a proposed structured classification, not VELOOP's internal codes. If VELOOP already has failure codes, they go in failure_detail_code and the categories are mapped to them.

failure_category	Definition	Typical Cause	How It Could Be Detected	Counts Against	Status
validation_failed	Completion was claimed but validation did not pass	Rules not met, or provider rejected	validation_status = failed	Validation pass rate	Proposed
tracking_failed	VELOOP cannot tell whether the user did the task	External tracking did not report back	No confirmation signal within an agreed wait time (length to be agreed)	Tracking reliability, not user behaviour	Proposed
user_abandoned	User started but did not finish	User left the flow	No claim and no activity within a timeout (length to be agreed)	Completion rate	Proposed
system_error	Failure inside VELOOP	Technical fault	Error recorded by backend	System reliability	Proposed
provider_error	External provider returned an error or was unavailable	Provider outage or bad response	Provider status or timeout	Provider reliability	Proposed
unknown	Failure with no known reason	Missing information	Default when nothing else applies	Data quality	Proposed

5. Reward events (C3)

Event	Meaning	Trigger	Required Properties	Linked To	Status
reward_pending	A reward is earned but not yet usable	Reward created and waiting for validation, review or provider confirmation	reward_id, user_id, reward_amount, reward_unit, source_feature, pending_reason, timestamp	task_activity_id	Proposed
reward_credited	Reward is actually added to the wallet	Wallet balance updated successfully	reward_id, user_id, reward_amount, reward_unit, source_feature, wallet_transaction_id, timestamp	wallet transaction	Proposed
reward_reversed	A credited reward was taken back	Operations or system undoes a credit	reward_id, original_reward_id, reward_amount, reward_unit, reason_category, wallet_transaction_id, timestamp	original reward, reversal transaction	Proposed
reward_corrected	The amount of a reward was changed after the fact	Operations or system adjusts the amount	reward_id, old_amount, new_amount, reward_unit, correction_reason, timestamp	original reward, adjustment transaction	Proposed
reward_rejected (name to be agreed)	A pending reward will never be credited	Pending reward refused or expired, if such a state exists	reward_id, user_id, reason_category, timestamp	pending reward	To Be Verified

6. Reward state transitions

Impossible transitions are listed because they become data quality checks in the later quality framework.

From	To	Allowed?	Note
(none)	pending	Yes	Reward waits for validation
(none)	credited	Yes	Instant credit where no waiting exists
pending	credited	Yes	Validation passed
pending	rejected	Yes, if this state exists	To be verified with engineering
credited	reversed	Yes	Original record is kept; reversal links to it
credited	corrected	Yes	Keep old and new amounts
reversed	credited	No	A new reward must be a new record, not a reopened one
credited	pending	No	Impossible state transition; flag as a data quality error
reversed	reversed	No	Duplicate reversal; flag as a data quality error

7. Activity occurred vs reward credited

Question	Measure With	Do Not Use	Why
How many tasks were completed?	task_completed (validated)	task_started or completion claims	A claim is not validated success
How many rewards were actually given?	reward_credited	task_completed	Activity occurred is not the same as reward was credited
How much was net earned?	Sum of reward_credited amount minus reward_reversed amount, adjusted by reward_corrected, per unit	A single sum across all units	Units may not be comparable; conversion rule needed (A002)
How many rewards are waiting?	reward_pending with no later credited or rejected	Tasks with no reward record	Pending is a recorded state; missing records are a data quality issue
How reliable are rewards?	reward_reversed / reward_credited	Reversals / tasks	Reversals relate to credits, not to activity

8. Third-party dependency (C4)

Dependency	What VELOOP May Not Control	Risk to Data	Fields to Keep	Analytics Approach
Advertisements	Whether an ad is served, finished and reported	Missing, late or inflated completion signals	provider_name, ad_placement_id, provider_reference_id, provider_status	Compare ad_started vs provider completion; treat missing signals as 'unconfirmed', not 'failed'
Surveys	Eligibility, screen-outs, completion confirmation	Completions confirmed days later; partial completions	provider_name, survey_id, provider_status, provider_reported_reward	Keep reward pending until provider confirms; track screen-out separately from failure
Offers	Partner conversion tracking and approval	Delayed approval, later reversal by partner	provider_name, offer_id, provider_status, approved_at	Allow long pending periods; expect reversals; report them separately
Games	Game-side progress and result reporting	Progress not reported or reported twice	provider_name, game_id, provider_reference_id, provider_status	Deduplicate by provider_reference_id
Tracking systems	Attribution and event delivery	Lost or duplicated callbacks, clock differences	provider_reference_id, callback_received_at	Store both the provider's time and VELOOP's receive time; reconcile regularly

Principles for externally driven earning

- Store the provider's own reference id on every externally driven activity so records can be matched.
- Keep VELOOP status and provider status in separate fields; do not overwrite one with the other.
- A missing provider signal is 'unconfirmed' until an agreed wait time passes (length to be agreed), then tracking_failed.
- Record both when the provider says it happened and when VELOOP received the signal.
- Reconcile provider reports against VELOOP records on a regular schedule (frequency to be agreed).
- Do not report provider-driven results as if VELOOP controlled them.

9. Open questions for verification

- Does the backend emit task completion and validation events today? (A001)
- Which reward units exist, and can they be summed or converted? (A002)
- Which providers are used, and what callbacks do they send? (A009)
- Is there a pending reward state, and can rewards be reversed or corrected today? (A013, A020)
- What wait times should define abandoned and tracking_failed? (stakeholder decision; no value is invented here)

Next: Work Package D - Wallet and Withdrawal Analytics.